AI Engineering and Claude Architect · Part III, Agents

Agents and MCP, Lesson 7

Sessions, Memory and Long-Running Harnesses

A team resumed yesterday's analysis session. Claude confidently described a function that a colleague had deleted that morning, because its old tool results were still in the conversation. Resuming restored the conversation, and not the world.

Module	AI06 of 13
Lesson	7 of 10
Prerequisites	AI03 L6, AI06 L2, AI06 L6
Exam relevance	**Direct.** All of TS 1.7, and parts of TS 5.4
Reading time	~40 minutes

Contents

1. Sessions, Memory and Long-Running Harnesses

Lesson 7 of 10 · Agents and MCP

Sessions, Memory and Long-Running Harnesses

Key idea

A session saves the conversation, not the filesystem, and it can't know what's changed since. So resume when earlier results are still valid, tell the agent what changed, and start fresh with a summary when too much is stale. **Long work survives on durable state outside the conversation: a progress file, a task list, git, a manifest.**

> *Before: context engineering, case facts and compaction (AI03 L6). Teaches: resume, continue and fork; resume vs fresh; Anthropic's harness pattern; manifests and scratchpads. Used by: AI06 L9–L10, AI12.*

The hook: stale results look exactly like facts

A tool result from yesterday looks identical to one from a second ago. Claude can't tell, unless you tell it.

Here's a small lab with a manifest on disk and file hashes. Real output from `code/ai06-agents/.../HarnessState.java`, with "analysis" simulated:

```
Run 1: simulated crash after 2 of 5 files
Run 2: analyzed 3 more, skipped the finished ones
One edit:   stale [Service2.java] -> resume, and name the changed files
Three edits: stale [Service2.java, Service3.java, Service4.java] -> start fresh, with a structured summary
```

The crash lost nothing, and the staleness check picked the strategy. The 30% threshold behind that choice is my engineering judgment for this example, and not a documented number.

Intuition: handing over a shift

A hospital nurse finishing a shift doesn't rely on memory, and doesn't pass along every conversation from the day.

Shift practice	Agent equivalent
Patient chart, updated as things happen	Progress file, manifest
Handover notes: what's changed, what to watch	Structured summary
"Check the lab results again, they're from this morning"	Name what's stale
A second doctor trying another treatment plan from the same notes	Fork

Anthropic's harness post uses this very image: engineers working in shifts with no handoff notes.

Mechanism: sessions

From the Agent SDK sessions docs: a session is the accumulated history, written to disk automatically. **It persists the conversation, not the filesystem.**

Operation	What it does	Claude Code CLI	Agent SDK
Continue	Picks up the most recent session in this directory	`--continue`	`continue` / `continue_conversation`
Resume	Picks up a specific session	`--resume <session-id or name>`	`resume`
Fork	New session starting from a **copy** of the history	`--fork-session` with resume	`forkSession` / `fork_session`

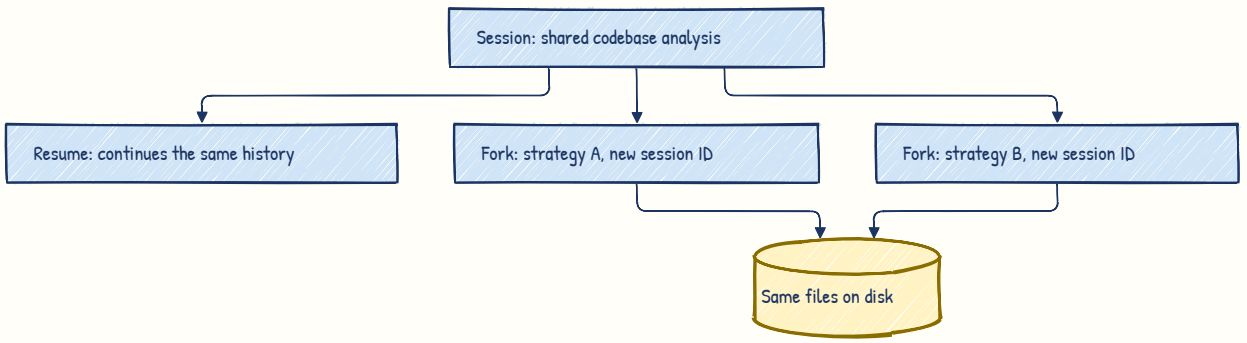


Figure 1. Resume adds to the same session. Fork copies the history into a new one, leaving the original unchanged. Both branches still share the same files on disk.

Forks don't isolate files

Forking branches the conversation, not the filesystem. If two forks edit files in one directory, those changes are real and visible to both. To branch files too, use git worktrees (AI12) or the SDK's file checkpointing.

Session files are local. To resume on another host, use a `SessionStore` adapter or move the file. Or, as the docs suggest, capture what you need as application state and pass it to a fresh session, which is "often more robust".

Resume or start fresh?

This is the core judgment in **TS 1.7**.

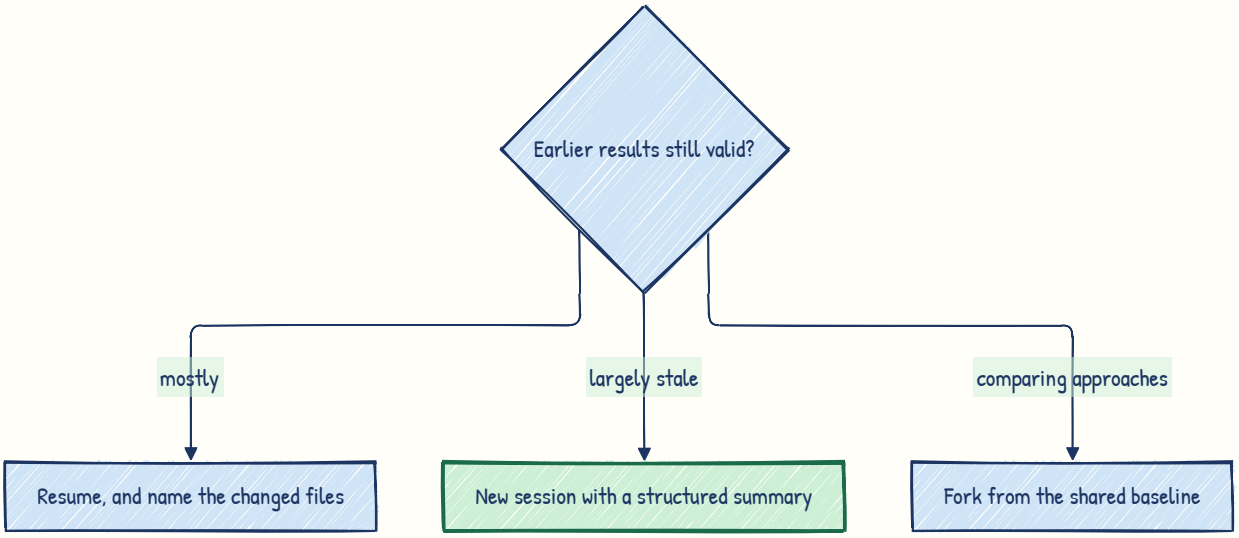


Figure 2. Choosing how to continue. The deciding question is whether earlier tool results still describe reality.

Situation	Choose	What to do
Follow-up with nothing changed	**Resume**	Ask directly
A few files edited	**Resume**	"`Service2.java` changed since you read it. Re-read it, and check your conclusions." Targeted, not a full re-exploration
Big refactor since then	**Fresh with summary**	Key decisions, file map, open questions
Two possible designs	**Fork**	One fork per design
Hit a turn or budget limit	**Resume**	With a higher limit

A structured summary beats stale tool results, since it carries decisions without carrying outdated evidence.

Spotting context degradation in a long exploration (TS 5.4)

Exam guide wording, and a symptom you should recognize: in an extended session, **the model starts giving inconsistent answers and refers to "typical patterns" instead of the specific classes it found earlier.** It isn't confused about programming. The specifics have left its working context.

Practice	What it does
Scratchpad file	The agent records key findings and rereads them for later questions, so they survive a context boundary
Subagents for narrow questions	"Find all test files" or "trace the refund flow's dependencies" runs elsewhere, and the main agent keeps the high-level view
Summarize, then spawn	Summarize one phase's findings, and inject that summary into the next phase's subagents as initial context
`/compact`	Reduces context use when verbose discovery output has filled the session

Managing context within a long run

AI03 L6 covers these fully. Here's how to choose:

Tool	Does what	Scope
Context editing	Clears old tool results	Within a session
Compaction	Summarizes earlier context	Within a session, near the limit
Memory tool (`memory_20250818`)	Files that persist across sessions	Across sessions
Subagents	Keep verbose work out of the main context	Within a task
Scratchpad and manifest files	State you control	Across sessions and crashes

Keep amounts, IDs and dates in a **case facts** block outside summarized history (TS 5.1). **TS 5.4** adds scratchpad files, subagents for verbose work, `/compact`, and manifests for crash recovery.

How Anthropic solved it

From [Effective Harnesses for Long-Running Agents](), November 2025.

The problem. Agents work in discrete sessions, and each new session starts with no memory.

Compaction alone wasn't enough: even Opus 4.5, run in a loop with the Agent SDK and only a high-level prompt, fell short of a production-quality app.

Two failure patterns:

1. The agent tries to one-shot the app, runs out of context mid-feature, and leaves it half-built and undocumented.
2. A later agent sees progress, and declares the job done.

The solution: two prompts, same harness. They note these are separate agents only in that they get different first prompts.

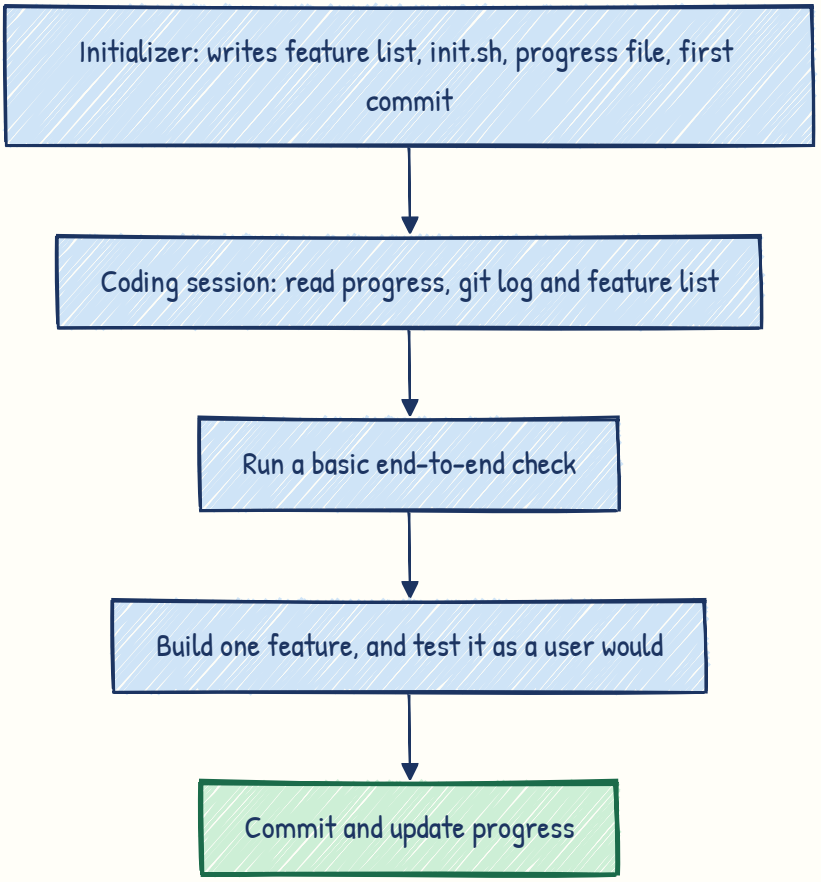


Figure 3. Anthropic's published harness. A first session sets up durable state, and every later session reads it, completes one feature, tests it, and leaves a clean handoff.

Artifact	Purpose	Detail
Feature list (JSON)	Stops early victory claims	Over 200 features for their claude.ai clone, all starting as failing. Agents may only change the `passes` field
`claude-progress.txt`	What each session did	With git history, a fresh context catches up quickly
Git	Recovery	Revert bad changes
`init.sh`	Start the app	No time lost working out how
One feature at a time	Counters over-reach	
Browser testing	Finds what unit tests and `curl` miss	Claude had marked features done without end-to-end checks

They chose **JSON because the model is less likely to inappropriately overwrite it than Markdown.** That's a nice example of designing around observed behavior.

Problem	Initializer does	Coding session does
Declares victory early	Creates the feature list	Works on one feature from the list
Leaves bugs, or no notes	Sets up git and notes	Reads notes, runs a check, commits
Marks features done too soon	Creates the feature list	Marks passing only after testing
Wastes time starting the app	Writes `init.sh`	Reads `init.sh`

What's still open, by their account: whether specialized testing or QA agents would beat one general agent, and how far this applies beyond web development.

Why the obvious fixes fall short

Tempting move	Why it fails	Better
Resume, and hope for the best	Stale results read as facts	Name what changed
Re-explore everything each time	Wasteful	Targeted re-analysis
Depend on compaction alone	Anthropic found it insufficient for long builds	Durable state
Keep it all in one ever-growing session	Context fills, and a crash loses it	Files and fresh sessions
Fork to isolate edits	Files are shared	Worktrees or checkpoints
Tell the agent "finish the whole app"	Runs out mid-feature	One feature per session

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: one long session. It works until the context fills or a process dies.

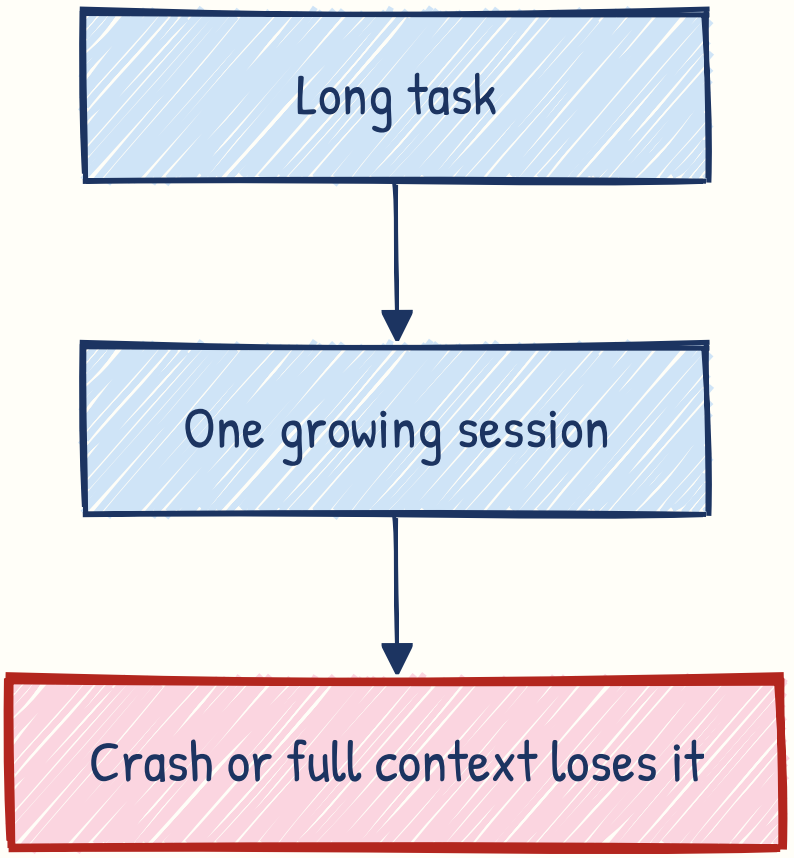


Figure 4. Baseline. All state lives in the conversation, so a crash or a full context loses the work.

Improved: saved transcripts with resume. A crash no longer loses the conversation. New problem: **resumed results may be stale, and the transcript is long and costly to carry.**

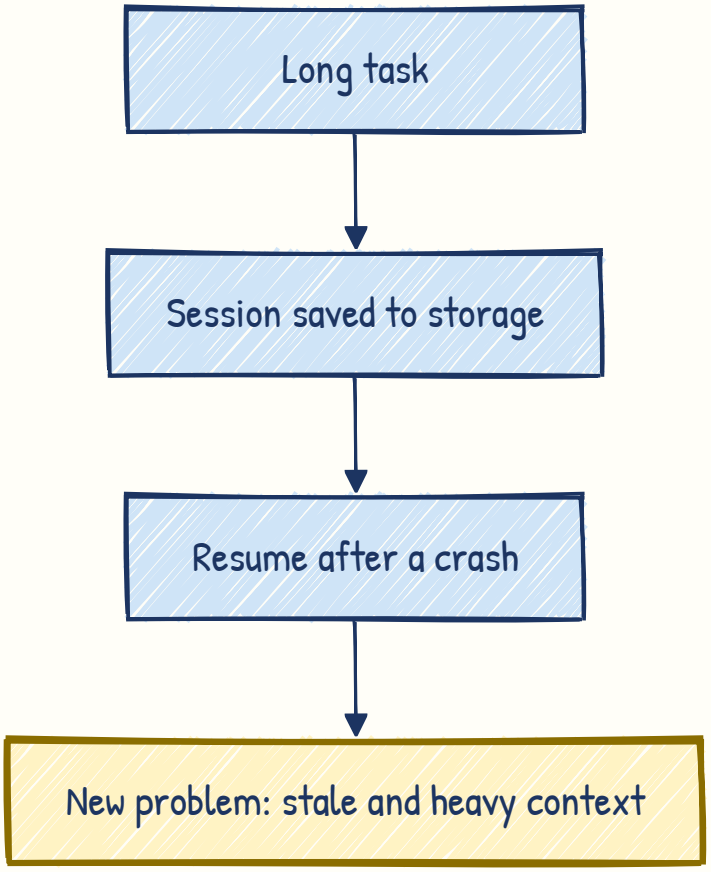


Figure 5. Improved. Resuming survives crashes, but carries outdated results forward.

Production: durable task state, with disposable sessions.

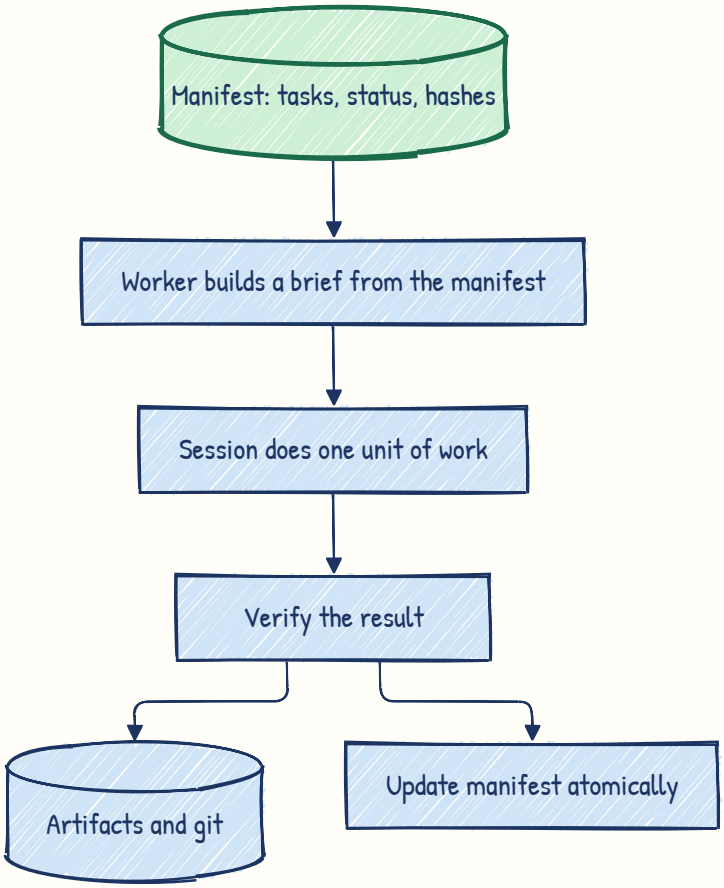


Figure 6. Production. The manifest and artifacts are the source of truth. Workers start with a short brief built from them, finish one unit, and record the result atomically.

Component	Why it exists	How it communicates
Manifest	Records what's done, for crash recovery	File or table, written atomically after each unit
Hashes	Detect staleness	Compared before reuse
Brief builder	Short, current context for each session	Reads manifest and artifacts
Verification	Stops false "done" claims	Tests, not the agent's opinion

Failures

Failure	Containment
Crash mid-write	Write to a temp file, then rename
Agent claims done falsely	Status changes only after verification
Agent rewrites the task list	Restrict edits to a status field, or apply them in code
Session file lost on another host	Session store, or rely on the manifest
Two workers on one task	Claim tasks with a lease

Java

From `HarnessState.java`. The write is atomic, and it happens after each unit, so a crash loses at most one.

```
Path temp = manifest.resolveSibling("manifest.tmp");
```

```
save(manifest, done, hashes); // after each task, and atomically
```

And the decision itself:

```
return (double) staleCount / total > FRESH_START_ABOVE
        ? "start fresh, with a structured summary"
        : "resume, and name the changed files";
```

A hash of file contents here uses `hashCode` for brevity. Use SHA-256 in real code. The Agent SDK's session options are for Python and TypeScript, so a Java service would own this state itself.

Trade-offs

Decision	Gain	Cost
Resume	Full context, fast	Stale risk, and heavier context
Fresh with summary	Clean and current	Summary may lose detail
Fork	Compare approaches from one baseline	Shared files, and more tokens
Manifest	Crash-safe, inspectable	Upfront design
One unit per session	Reliable progress	More sessions, and more start-up cost

When you don't need this: tasks that finish within one short session.

Common mistakes

1. **Thinking resume restores the world.** It restores only the conversation.
2. **Not naming changed files.** Claude won't know.
3. **Re-exploring everything.** Be targeted.
4. **Expecting forks to isolate files.** They don't.
5. **Relying on compaction for very long work.** Keep durable state.
6. **Letting an agent mark its own success.** Verify.
7. **Saving the manifest only at the end.** A crash loses it all.

What to say in an interview

"A session persists the conversation, not the files. If results are mostly valid, I'd resume and name what changed. If they're largely stale, I'd start fresh with a structured summary. Fork when comparing approaches from a shared analysis, remembering files are shared. For long work I'd follow Anthropic's harness pattern: a task list, a progress file, git, and one verified unit per session."

Certification lens

How this lesson maps to CCAR-F

TS 1.7, and **TS 5.4.** Use `--resume <name>` for valid context, `fork_session` for branches, and a fresh session with a structured summary when tool results are stale. Tell a resumed session which files changed.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Code Generation with Claude Code · Medium. You analyzed a module yesterday in a named session. Today, two of its 40 files changed. What's the best approach?

A. Resume the session, and tell it which two files changed for targeted re-analysis.

B. Resume, and say nothing, because Claude will notice.

C. Start fresh, and re-explore all 40 files.

D. Fork the session, and edit the old transcript.

A **A** (TS 1.7). B leaves stale results standing. C is wasteful, as context is mostly valid. D isn't a real workflow.

Q Q2 · Exam style, scenario: Code Generation with Claude Code · Medium. A large refactor merged overnight, touching most files your session analyzed. What's the most reliable way to continue?

A. Resume, and ask Claude to be careful.

B. Run `/compact`, then continue in the same session.

C. Start a new session, with a structured summary of decisions and open questions.

D. Fork the old session.

A **C** (TS 1.7). When results are largely stale, fresh with a summary is more reliable. A and D carry stale results. B summarizes them.

Q Q3 · Exam style, scenario: Developer Productivity with the Agent SDK · Medium. You want to compare two testing strategies, starting from one shared codebase analysis. What should you use?

A. Two unrelated new sessions, each repeating the analysis.

B. One session, switching strategies midway.

C. A subagent that picks a strategy at random.

D. `fork_session`, to create a branch for each strategy from the analysis.

A **D** (TS 1.7, TS 1.3). A repeats the work. B mixes the strategies. C compares nothing. Note that forks share files on disk.

Q Q4 · Exam style, scenario: Code Generation with Claude Code · Medium. A long codebase exploration keeps losing earlier findings as context fills up, and a crash lost a day's work. Which approach best fixes both?

A. A bigger context window.

B. Write findings to scratchpad files and a task manifest, and use subagents for verbose exploration.

C. Ask Claude to remember important findings.

D. Disable compaction.

A **B** (TS 5.4). A is a bigger hammer that doesn't survive a crash. C isn't durable. D makes it worse.

Q Q5 · Case study · Hard. An overnight coding agent keeps marking features "done" that fail in the browser, then ends early, claiming the project is finished. Using Anthropic's published harness, give three changes.

A (1) A JSON feature list with all items failing, where agents may change only `passes`. (2) One feature per session, starting with reading the progress file and git log, and running a basic end-to-end check. (3) Explicit browser testing, as a user would, before marking anything as passing. Finish each session with a commit and a progress update.

Revision - Lesson 7

→ A session persists the conversation, not the filesystem.

→ Continue: latest here. Resume: a specific session. Fork: a new session from a copy.

→ Forks share files on disk.

→ Mostly valid: resume, and name what changed.

→ Largely stale: new session, with a structured summary.

→ Tell a resumed session which files changed, for targeted re-analysis.

→ Compaction alone wasn't enough for long builds, per Anthropic.

→ Anthropic's harness: initializer, then coding sessions with durable artifacts.

→ Artifacts: JSON feature list, progress file, git, init.sh.

→ One feature per session, tested as a user would.

→ Manifests make crashes cheap. Write atomically after each unit.

→ Verification decides done, and the agent's opinion doesn't.

Your notes